5주차 강의

지능형 IoT 네트워크

지능형 IoT 데이터 수집

김한진
gks359@cbnu.ac.kr
충북대학교 산업인공지능연구센터

INDUSTRIAL AI RESEARCH CENTER

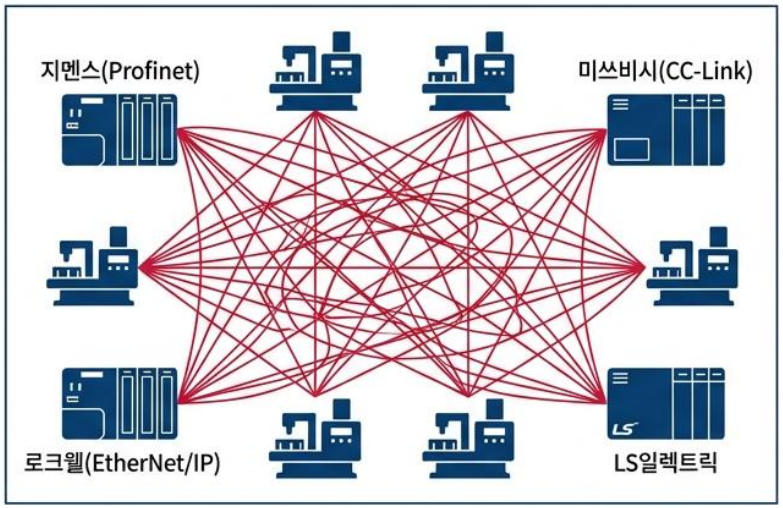
CONTENTS

I 미들웨어

II MQTT

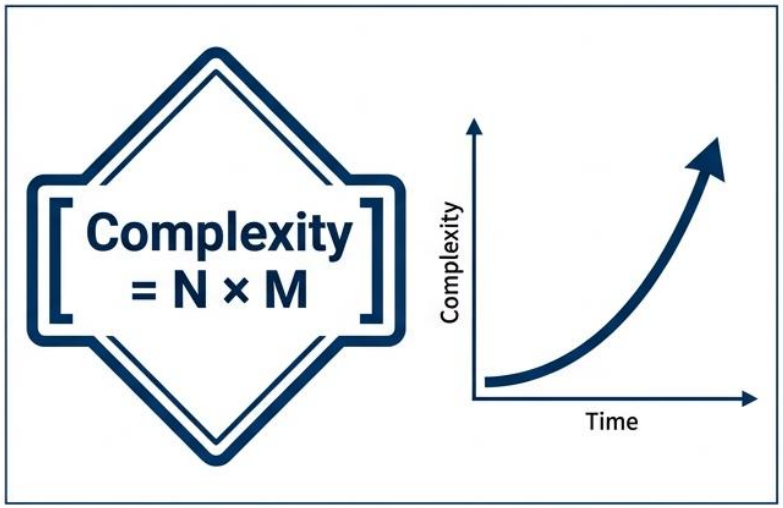
■ 미들웨어

➢ 서로 다른 제조사 장비들을 1:1로 직접 연결하면, 유지보수가 불가능한 '스파게티 연결'



제조 현장의 현실 - 프로토콜 사일로

- 하드웨어 종속 : 특정 프로토콜에 묶여 설비 확장 제한
- 데이터 포맷 불일치 : 장비 간 데이터 포맷 불일치로 호환 어려움

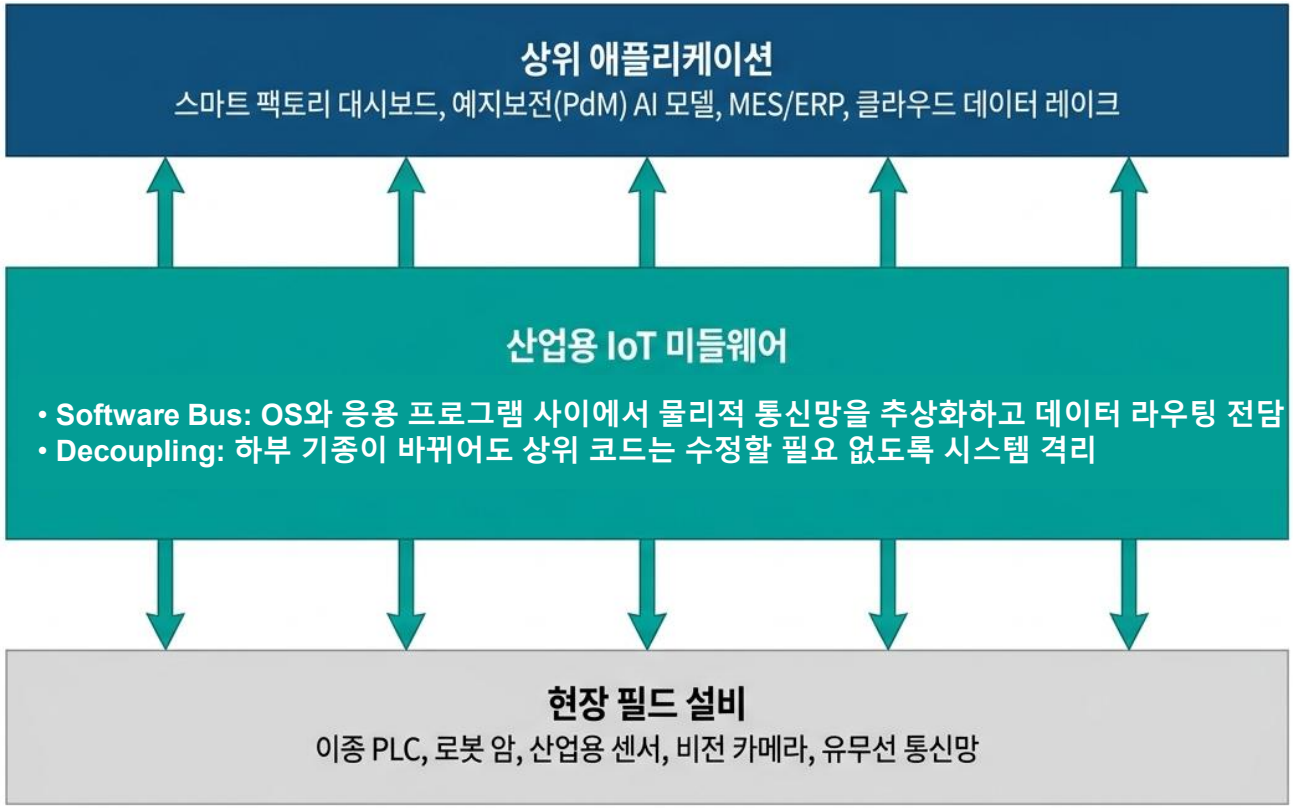


결과 및 비즈니스 손실

- 점대점 한계 : 설비 1대를 교체하면 상위 MES, SCADA 등 코드 전체를 고쳐야 함
- 데이터 사일로 : 사내 데이터가 특정 설비와 벤더의 폐쇄망 안에 갇히는 현상 심화

■ 미들웨어

➢ 미들웨어는 하부 설비의 복잡성을 흡수, 상위 시스템에 일관된 데이터 창구 제공

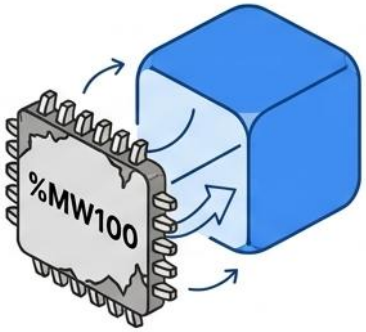


INDUSTRIAL AI RESEARCH CENTER

■ 미들웨어

➢ 미들웨어의 핵심 기능

Card 1. 하드웨어 추상화
(Abstraction)



복잡한 16진수 메모리 주소(%MW100)를
몰라도, '1호기 프레스의 가압력'이라는
객체명으로 접근하도록 래핑

Card 2. 데이터 정규화
(Semantic Transformation)



윈시 바이너리(Raw Binary)를
산업 표준 JSON/객체 포맷으로 변환
타임스탬프 동기화 및
계측 단위(psi → bar) 자동 표준화

Card 3. 메시지 큐잉 및 라우팅
(Buffering & Routing)

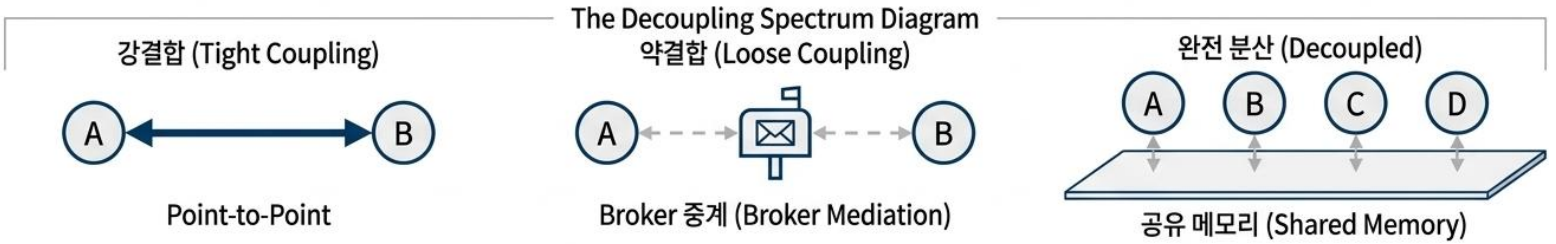


생산자-소비자 간 데이터 처리 속도
불일치 해결
통신 단절 시 링 버퍼(Ring Buffer)에
데이터 보관하여 유실 방지

■ 미들웨어

➢ 통신 패러다임의 비교, RPC vs MOM vs Data-Centric

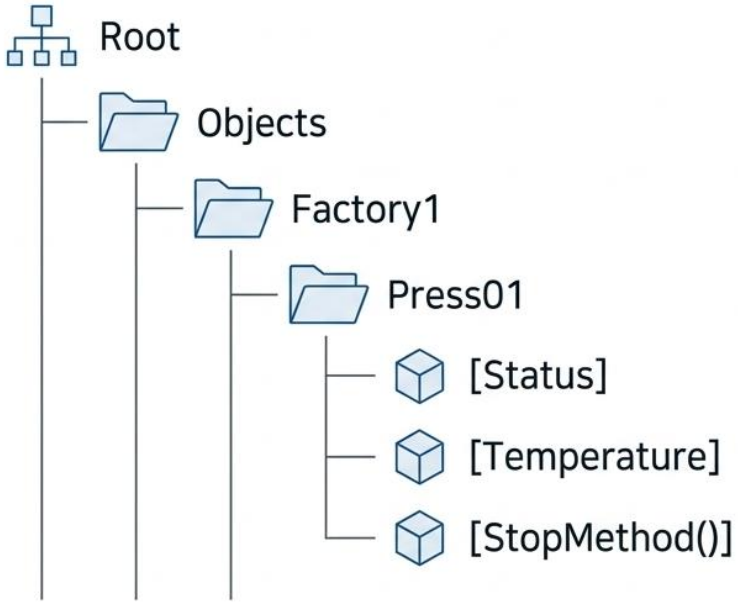
RPC : Remote Procedure Call / MOM : Message Oriented Middleware



구분	RPC/REST (원격 호출)	MOM (메시지 지향)	Data-Centric (데이터 중심)
대표 기술	gRPC, HTTP	MQTT, RabbitMQ	OMG DDS, ROS 2
통신 메커니즘	요청-응답 (Request-Response)	우체통 브로커 (Pub/Sub)	글로벌 데이터 공간 (P2P Bus)
시간 결합도	동기식(Sync) - 응답 대기	비동기(Async) - 즉시 다음 작업	완전 분산 - 실시간 공유
공간 결합도	상대방 IP/Port 필수	브로커 IP만 필요	IP 불필요 (데이터 타입 기반 매칭)
현장 적용처	정적 환경설정 조회	무선 센서 수집/알람	자율주행 AMR, 다축 로봇 동기화

■ 미들웨어

➢ OPC UA는 단순한 통신 규약을 넘어, 설비의 메타데이터를 담아내는 '객체 지향 모델링' 표준

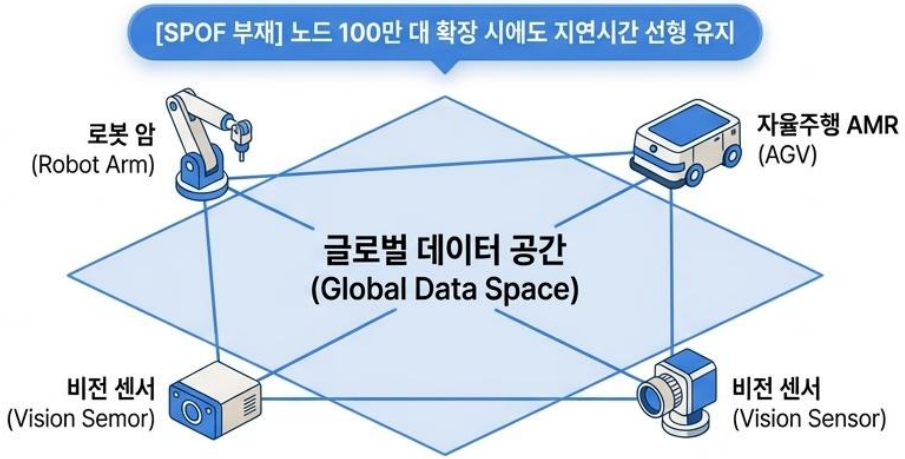


- **통신 vs 정보 모델링**: 기존 Modbus(40001 = 235)는 데이터의 의미를 알 수 없으나, OPC UA는 설비명, 단위(℃), 정상 오차 범위, 점검 이력 등 메타데이터를 **제공**
- **자기 기술적**(Self-describing) 구조: AI 모델이 서버에 접속하여 설비 노드 트리를 직접 검색(Browse)하고 필요한 센서 값을 즉시 탐색 가능.
- **플랫폼 독립성 및 보안**: 리눅스/임베디드 완벽 지원 및 X.509 인증서를 통한 통신 구간 내장 암호화.

https://www.youtube.com/watch?v=-tDGzwsBokY

■ 미들웨어

➢ OMG DDS는 중앙 서버의 병목을 제거, 노드 간 마이크로초 단위의 통신 실현



Brokerless P2P 아키텍처

단일 장애점(SPOF)이 없으며,
모든 참여 노드가 가상의
'글로벌 데이터 공간'을 공유하며 직접 통신.

22가지 극한의 QoS 제어

Deadline (정해진 시간 내 패킷 미도달 시 알람),
Reliability, Durability 등 미세 제어 가능.

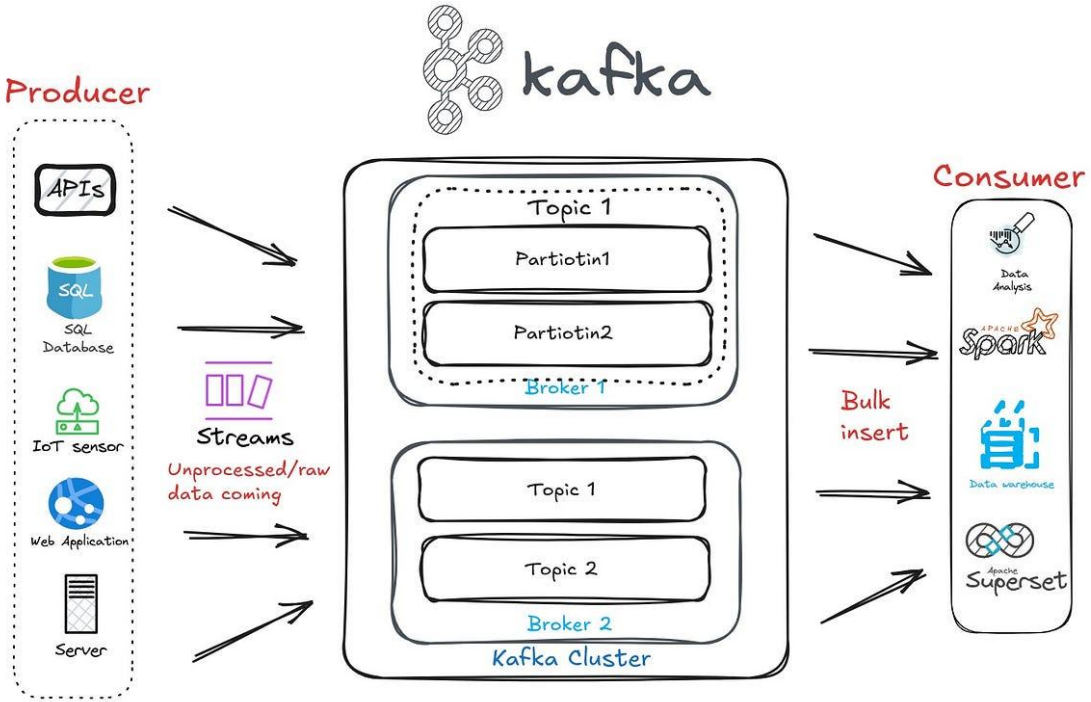
산업 적용처

ROS 2 (로봇 운영체제) 기본 미들웨어.
전 세계 자율주행 AMR, 협동로봇,
고속 패키징 라인, 방위산업.

https://www.youtube.com/watch?v=_cw4TuUH_Po

■ 미들웨어

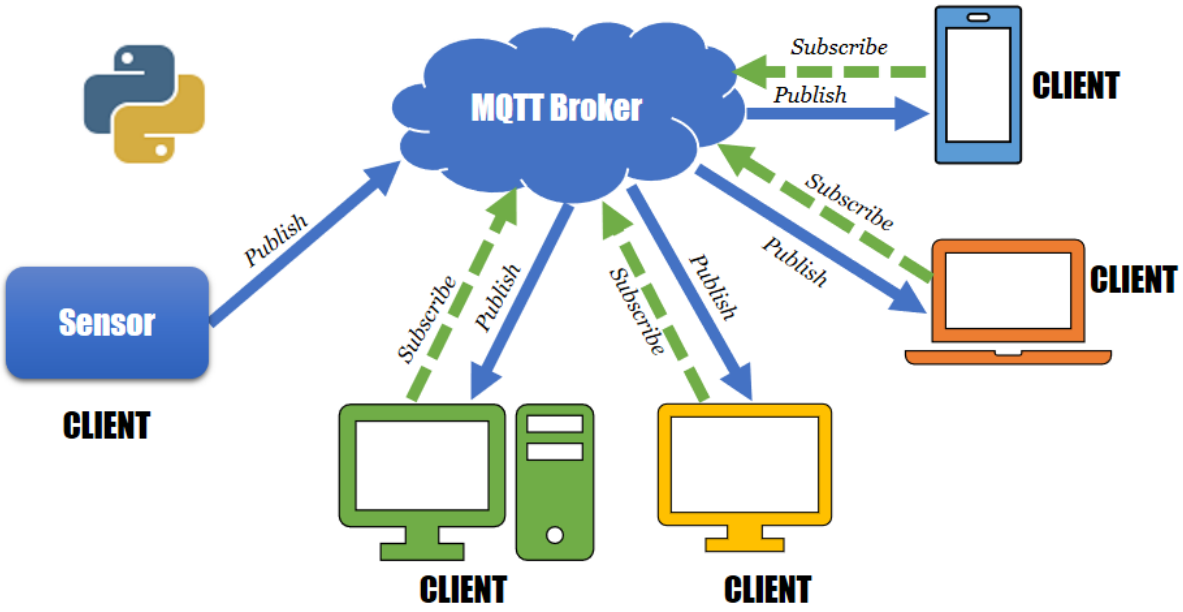
➢ Apache Kafka는 수많은 기업에서 고성능 데이터 파이프라인, 스트리밍 분석, 데이터 통합 및 미션 크리티컬 애플리케이션에 사용하는 오픈 소스 분산 이벤트 스트리밍 플랫폼



https://www.youtube.com/watch?v=Nh3AZx8D9KY

■ 미들웨어

- MQTT(Message Queuing Telemetry Transport)는 저대역폭 환경에서 IoT 원격 측정을 위해 설계된 경량 메시지 전송 프로토콜 (ISO 표준: ISO/IEC 20922)
- 헤더가 작게는 단 2바이트일 정도로 리소스를 매우 적게 요구



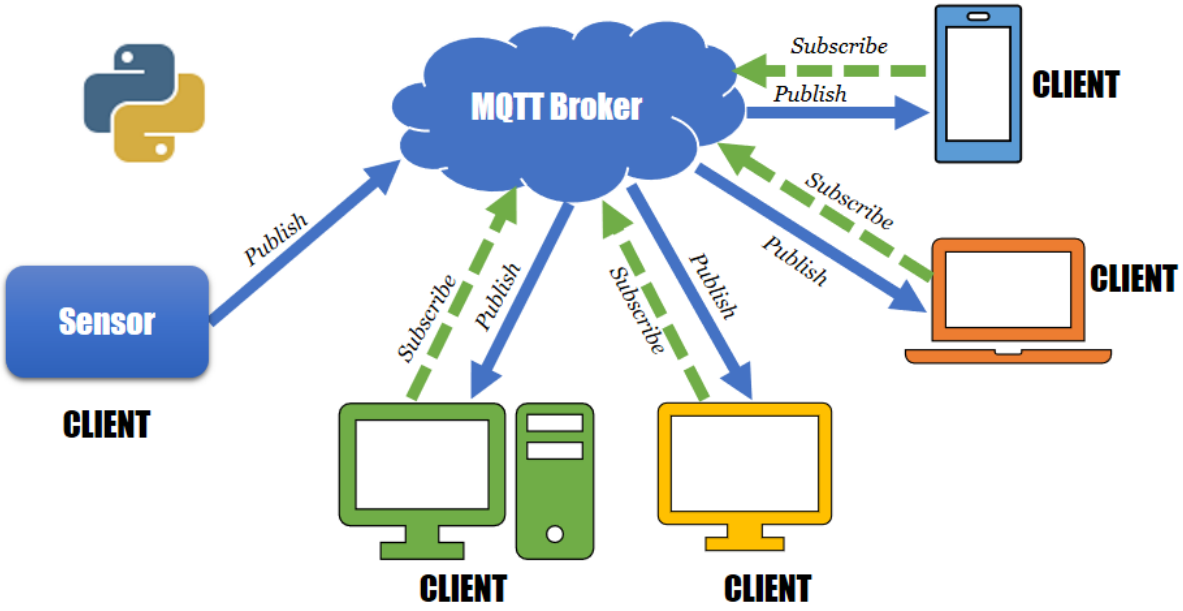
■ 미들웨어

➢ 미들웨어 기술 비교

비교 항목	OPC UA	OMG DDS	Apache Kafka	MQTT
통신 구조	클라이언트-서버 & Pub/Sub	분산 P2P (Brokerless)	분산 로그 브로커	중앙 중계 브로커
핵심 강점	표준 정보 모델링, 기기 호환성	초저지연 (Microsecond), 실시간성	대용량 분산 큐, 무손실 데이터 보관	초경량 헤더, 저전력, 무선 최적화
지연시간	수십 ms	**수 µs ~ 수 ms**	수 ms ~ 수십 ms	수 ms ~ 수십 ms
페이로드 구조	객체 지향 복합 데이터 노드	엄격한 IDL 타입 정의	바이너리 (Avro, JSON)	자유 바이너리 /JSON
현장 최적 영역	공장 내 PLC/SCADA 연동	로봇 제어, 자율주행 AMR	전사 데이터 파이프라인, AI 백본	무선 센서 수집, 원격 자산 관제

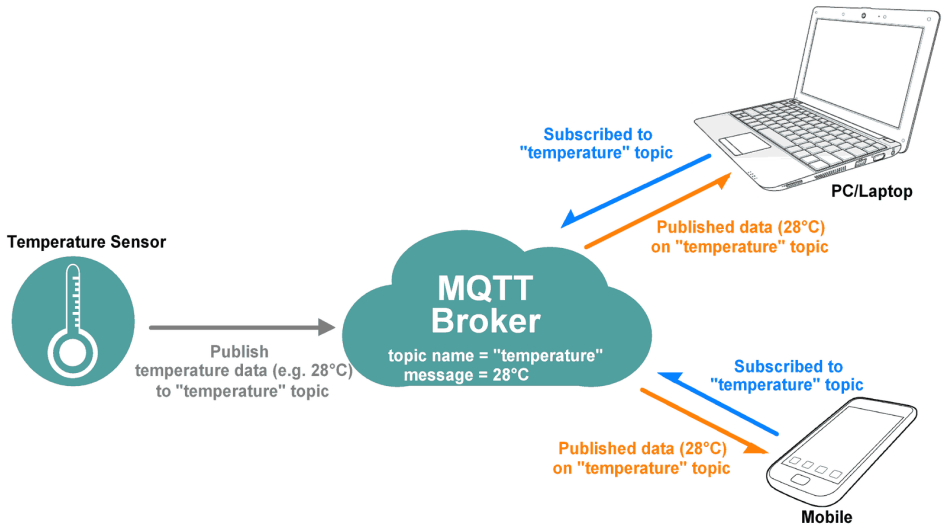
MQTT란?

- Message Queuing Telemetry Transport)는 저대역폭 환경에서 IoT 원격 측정을 위해 설계된 경량 메시지 전송 프로토콜 (ISO 표준: ISO/IEC 20922)
- 헤더가 작게는 단 2바이트일 정도로 리소스를 매우 적게 요구



MQTT란?

- 통신은 '브로커'라고 불리는 서버에 의해 중계
- 클라이언트들은 임의의 '토픽'을 '구독(subscribe)'하거나 토픽에 메시지를 '게시(publish)'하는 식으로 상호간 통신을 수행



MQTT Quality of Service (QoS)

- QoS는 데이터 전송의 품질을 보장하기 위한 기능 (좁은 의미)
- MQTT에서는 메시지가 수신자에게 도달하는 보장 수준을 정의
  단, 네트워크 layer에서 해당 QoS 수준을 만족시키지 못하면 지원 불가

QoS 레벨	전송 보장 수준	메시지 손실 위험	중복 수신 위험	사용 예
0	최대 한 번 (At most once)	있음	없음	센서 데이터(모니터링) 등 중요도가 낮은 메시지
1	최소 한 번 (At least once)	낮음	있음 (중복 가능)	제어 신호 등 중요도가 높은 메시지
2	정확히 한 번 (Exactly once)	없음	없음	금융/거래 정보 등 중복되면 안 되는 메시지

MQTT 구성 (실시간 센서 데이터 모니터링 시스템)

- MQTT Broker (중앙 서버): 메시지 송수신 중계
- Publisher (발행자): 데이터를 보내는 장치 (예: 센서)
- Subscriber (구독자): 데이터를 받는 장치 (예: 웹서버)

구성 요소	역할	설명
Mosquitto	Broker	중계 서버. Publisher와 Subscriber 사이의 통신 관리
Paho MQTT	클라이언트	Python으로 MQTT 기능 구현

MQTT Broker - Mosquitto

- Eclipse 재단의 오픈소스 MQTT 메시지 브로커
- MQTT 메시지를 중계하는 서버 역할
- Publisher와 Subscriber 간 통신 중재

➢ 설치

```
$ sudo apt install mosquitto mosquitto-clients
```

```
$ sudo systemctl enable mosquitto
```

```
$ sudo systemctl start mosquitto
```

설치 링크 : https://mosquitto.org/blog/2013/01/mosquitto-debian-repository/

MQTT Mosquitto test

- Publisher

```
$ mosquitto_pub -h localhost -t "test/topic" -m "hello mqtt"
```

- Subscriber

```
$ mosquitto_sub -h localhost -t "test/topic"
```

감사합니다

Q&A